

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)

Отчет

О выполнении лабораторной работы №2

«Сверточные глубокие сети»

По дисциплине «Нейросетевые технологии искусственного интеллекта»

Вариант задания №2

Выполнила студентка группы М8О-109СВ-25

Бондарева Е.Е. _____

Проверил и принял

Доцент каф. 802 Леонов С.С. _____

С оценкой _____

Лабораторная работа №2

Сценарий работы

Этап 1. Построение сверточной нейронной сети для распознавания объектов из базы данных MNIST.

Целью данного этапа лабораторной работы является создание сверточного многокатегориального классификатора изображений цифр из набора данных MNIST. Пошаговая реализация поставленной цели включает:

1. Загрузку набора данных MNIST;
2. Разделение данных на обучающий и тестовый наборы;
3. Подготовку данных для передачи в нейронную сеть;
4. Выполнить конструирование сети: создание сверточной основы в соответствии с вариантом, добавить к сверточной основе классификатор заданной архитектуры, вид активационной функции на сверточном слое и в скрытых слоях классификатора выбрать *relu*, самостоятельно выбрать количество и расположение слоев субдискретизации;
5. Настройку оптимизатора с выбором функции потерь и метрики качества. Число эпох принять от 30 до 50;
6. Проведение проверки решения, выделяя контрольное множество;
7. Вывод графиков функции потерь и точности;
8. Использование обученной сети для предсказания на новых данных.

Таблица 1.1 Вариант задания для первого этапа

№	Количество слоев	Количество карт признаков на слое	Размер ядра свертки	Количество скрытых слоев классификатора (количество нейронов на слое)
2	2	32-64	3x3	1 (256)

Таблица 1.2 Вариант для базовой сети

№	Количество слоев	Количество карт признаков на слое	Размер ядра свертки	Количество скрытых слоев классификатора (количество нейронов на слое)
1	2	32-64	3x3	1 (64)

Провести сопоставление результатов, полученных построенной моделью, с результатами базовой сети.

Этап 2. Построение бинарного классификатора. Обучение на малом объеме данных.

Целью данного этапа лабораторной работы является создание бинарного классификатора изображений кошек и собак из набора данных Cats vs. Dogs. Пошаговая реализация поставленной цели включает:

1. Загрузку набора данных Cats vs. Dogs;
2. Разделение данных на обучающий и тестовый наборы;
3. Создание генератора изображений заданной размерности; 4. Подготовку данных для передачи в нейронную сеть;
5. Выполнить конструирование сети: создание сверточной основы в соответствии с вариантом, добавить к сверточной основе классификатор заданной архитектуры, вид активационной функции на сверточном слое и в скрытых слоях классификатора выбрать relu, самостоятельно выбрать количество и расположение слоев субдискретизации;
6. Настройку оптимизатора с выбором функции потерь и метрики качества. Число эпох принять от 30 до 50;
7. Проведение проверки решения, выделяя контрольное множество;
8. Вывод графиков функции потерь и точности;
9. Использование обученной сети для предсказания на новых данных.

Таблица 1.3 Вариант задания для второго этапа

№	Размер входного изображения	Количество слоев	Количество карт признаков на слое	Размер ядра свертки	Количество скрытых слоев классификатора (количество нейронов на слое)
2	150x150	3	32-64-128	3x3	1 (512)

Таблица 1.4 Вариант для базовой сети

№	Размер входного изображения	Количество слоев	Количество карт признаков на слое	Размер ядра свертки	Количество скрытых слоев классификатора (количество нейронов на слое)
1	150x150	4	32-64-128-128	3x3	1 (512)

Провести сопоставление результатов, полученных построенной моделью, с результатами базовой сети.

Этап 3. Использование предварительно обученной сети.

Целью данного этапа лабораторной работы является использование предварительно обученной на наборе Imagenet нейронной сети VGG16 (или иной предобученной сетью) для увеличения точности классификации изображений из набора данных Cats vs. Dogs. Пошаговая реализация поставленной цели включает:

1. Загрузку набора данных Cats vs. Dogs и сеть VGG16 (или любую другую сеть по выбору) с весовыми коэффициентами, полученными при обучении на наборе данных Imagenet;
2. Разделение данных на обучающий и тестовый наборы;
3. Создание генератора изображений заданной размерности;
4. Подготовку данных для передачи в нейронную сеть;
5. В соответствие с вариантом сконструируйте сеть, используя вариант классификатора (слои расположенные после сверточной основы) из этапа 2;
6. Настройку оптимизатора с выбором функции потерь и метрики качества, используя количество эпох обучения согласно варианту;
7. Проведение проверки решения, выделяя контрольное множество;
8. Вывод графиков функции потерь и точности;
9. Использование обученной сети для предсказания на новых данных;
10. При ограниченных вычислительных ресурсах можно не проводить полного обучения на всех эпохах, а остановиться после наступления переобучения или стабилизации точности.

Таблица 1.5 Вариант задания для третьего этапа

№	Использование обученной сети	Количество эпох обучения
2	Выделение признаков обучающего набора при помощи сверточной основы с последующим их использованием как входов классификатора	40

Проведите сопоставление результатов этапов 2 и 3.

Этап 1. Построение сверточной нейронной сети для распознавания объектов из базы данных MNIST.

Импорт библиотек:

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.model_selection import train_test_split

import ssl
import urllib
```

Загрузка и преобработка данных:

На данном этапе выполняется загрузка набора данных MNIST, содержащего изображения рукописных цифр. Для корректного скачивания данных временно отключается проверка SSL-сертификата. Набор данных автоматически разделен на обучающую и тестовую выборки. Обучающая выборка содержит 60 000 изображений размером 28×28 пикселей, тестовая — 10 000 изображений такого же размера. После загрузки проверяются размерности полученных массивов данных.

```
ssl._create_default_https_context = ssl._create_unverified_context
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
ssl._create_default_https_context = ssl.create_default_context
```

```
print(f"Размер обучающей выборки: {x_train.shape}")
print(f"Размер тестовой выборки: {x_test.shape}")
```

Результат выполнения:

```
Размер обучающей выборки: (60000, 28, 28)
Размер тестовой выборки: (10000, 28, 28)
```

Нормирование данных:

Выполняется предварительная обработка данных для подготовки к обучению нейронной сети. Пиксельные значения изображений нормируются в диапазон $[0, 1]$ путём деления на 255, что улучшает сходимость алгоритма оптимизации. К данным добавляется измерение канала, преобразуя тензоры из формата (samples, 28, 28) в (samples, 28, 28, 1), что соответствует ожидаемому входному формату свёрточных слоев. Метки классов преобразуются в one-hot encoding с помощью функции `to_categorical`, создавая бинарные векторы длины 10. Обучающая выборка дополнительно разделяется на тренировочную и валидационную части в соотношении 75%/25% для мониторинга процесса обучения и предотвращения переобучения.

```
# Нормирование данных
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# Добавляем ещё одно измерение (канал)
x_train = np.expand_dims(x_train, -1)
x_test = np.expand_dims(x_test, -1)

# Разбиваем на категории (10 так как предсказываем цифры)
y_train = keras.utils.to_categorical(y_train, 10)
y_test = keras.utils.to_categorical(y_test, 10)

# Разделение данных на тренировочную и валидационную выборки
x_train, x_val, y_train, y_val = train_test_split(x_train, y_train,
test_size=0.25, random_state=42)
```

Построение и обучение моделей:

Создается архитектура сверточной нейронной сети для классификации изображений цифр. Модель построена по последовательной схеме и включает два сверточных слоя с ядрами 3×3 и активацией `relu`, чередующихся с слоями макс-пулинга для постепенного уменьшения пространственной размерности. Первый сверточный слой извлекает 32 карты признаков, второй — 64. После сверточной основы следует слой `Flatten`, преобразующий двумерные карты признаков в одномерный вектор, который передается в полносвязные слои. Классификатор состоит из одного скрытого слоя с 256 нейронами и выходного слоя с 10 нейронами и активацией `softmax` для многоклассовой классификации.

```
# Основная модель
model = keras.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)), # Слой
    свертки
    layers.MaxPooling2D((2, 2)), # Пулинг слой. Для уменьшения размерности карты
    признаков

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(), # Преобразует двухмерную карту признаков в одномерный
    вектор

    layers.Dense(256, activation='relu'), # Полносвязный слой

    layers.Dense(10, activation='softmax')
])

model.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
conv2d_7 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_7 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_8 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_8 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_3 (Flatten)	(None, 1600)	0
dense_7 (Dense)	(None, 256)	409,856
dense_8 (Dense)	(None, 10)	2,570

Total params: 431,242 (1.65 MB)

Trainable params: 431,242 (1.65 MB)

Non-trainable params: 0 (0.00 B)

Далее производится компиляция модели перед началом обучения. В качестве оптимизатора выбран алгоритм Adam, который сочетает преимущества методов RMSProp и Momentum, обеспечивая эффективное обновление весовых коэффициентов. Для задачи многоклассовой классификации используется функция потерь `categorical_crossentropy`, которая оптимально подходит для работы с one-hot encoded метками. В качестве метрики качества выбрана ассурасу (точность), показывающая долю правильных предсказаний модели на обучающих и проверочных данных.

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

Затем создается базовая архитектура сверточной нейронной сети для решения задачи классификации изображений MNIST. Модель состоит из двух сверточных блоков, каждый из которых включает слой свертки с ядром 3×3 и активацией `relu`, followed by слоем макс-пулинга с окном 2×2 для постепенного уменьшения пространственной размерности и выделения наиболее значимых

признаков. После сверточных слоев выполняется преобразование данных в одномерный вектор с помощью слоя Flatten, который передается на вход полносвязной части сети. Классификатор содержит один скрытый слой с 64 нейронами и выходной слой с 10 нейронами и активацией softmax для распределения по классам.

```
# Базовая модель
base_model = keras.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),

    layers.Dense(64, activation='relu'),

    layers.Dense(10, activation='softmax')
])

model.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
conv2d_7 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_7 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_8 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_8 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_3 (Flatten)	(None, 1600)	0
dense_7 (Dense)	(None, 256)	409,856
dense_8 (Dense)	(None, 10)	2,570

Total params: 431,242 (1.65 MB)

Trainable params: 431,242 (1.65 MB)

Non-trainable params: 0 (0.00 B)

Аналогично, как и для основной модели, производится компиляция модели перед началом обучения.

```
base_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

Выполняется процесс обучения сверточной нейронной сети. Обучение проводится в течение 15 эпох с размером мини-батча 128 образцов. На каждой итерации модель получает пакеты данных, вычисляет градиенты и обновляет весовые коэффициенты. Для контроля переобучения и оценки обобщающей способности модели используется валидационная выборка, на которой вычисляются метрики качества после каждой эпохи. Параметр `verbose=1` обеспечивает подробный вывод процесса обучения с отображением прогресса и метрик для каждой эпохи.

```
epochs = 15
batch_size = 128
history = model.fit(
    x_train, y_train,
    batch_size=batch_size,
    epochs=epochs,
    validation_data=(x_val, y_val),
    verbose=1
)
```

Результат работы:

```
Epoch 1/15
352/352 ————— 9s 23ms/step - accuracy: 0.9321 - loss: 0.2303 -
val_accuracy: 0.9740 - val_loss: 0.0836
Epoch 2/15
352/352 ————— 8s 23ms/step - accuracy: 0.9821 - loss: 0.0585 -
val_accuracy: 0.9837 - val_loss: 0.0549
Epoch 3/15
352/352 ————— 9s 24ms/step - accuracy: 0.9883 - loss: 0.0388 -
val_accuracy: 0.9876 - val_loss: 0.0405
Epoch 4/15
352/352 ————— 9s 24ms/step - accuracy: 0.9910 - loss: 0.0282 -
val_accuracy: 0.9877 - val_loss: 0.0438
Epoch 5/15
352/352 ————— 8s 23ms/step - accuracy: 0.9928 - loss: 0.0222 -
val_accuracy: 0.9886 - val_loss: 0.0392
Epoch 6/15
352/352 ————— 8s 24ms/step - accuracy: 0.9950 - loss: 0.0162 -
val_accuracy: 0.9887 - val_loss: 0.0388
```

```

Epoch 7/15
352/352 ————— 9s 24ms/step - accuracy: 0.9961 - loss: 0.0129 -
val_accuracy: 0.9874 - val_loss: 0.0466
Epoch 8/15
352/352 ————— 9s 25ms/step - accuracy: 0.9962 - loss: 0.0110 -
val_accuracy: 0.9883 - val_loss: 0.0411
Epoch 9/15
352/352 ————— 9s 26ms/step - accuracy: 0.9970 - loss: 0.0099 -
val_accuracy: 0.9894 - val_loss: 0.0446
Epoch 10/15
352/352 ————— 9s 25ms/step - accuracy: 0.9979 - loss: 0.0068 -
val_accuracy: 0.9897 - val_loss: 0.0440
Epoch 11/15
352/352 ————— 9s 26ms/step - accuracy: 0.9980 - loss: 0.0066 -
val_accuracy: 0.9899 - val_loss: 0.0418
Epoch 12/15
352/352 ————— 10s 26ms/step - accuracy: 0.9990 - loss: 0.0036 -
val_accuracy: 0.9874 - val_loss: 0.0540
Epoch 13/15
352/352 ————— 9s 26ms/step - accuracy: 0.9969 - loss: 0.0089 -
val_accuracy: 0.9896 - val_loss: 0.0432
Epoch 14/15
352/352 ————— 9s 25ms/step - accuracy: 0.9988 - loss: 0.0038 -
val_accuracy: 0.9893 - val_loss: 0.0462
Epoch 15/15
352/352 ————— 9s 26ms/step - accuracy: 0.9982 - loss: 0.0058 -
val_accuracy: 0.9886 - val_loss: 0.0516

```

Аналогично для базовой модели.

```

Epoch 1/15
352/352 ————— 8s 22ms/step - accuracy: 0.9182 - loss: 0.2874 -
val_accuracy: 0.9721 - val_loss: 0.0937
Epoch 2/15
352/352 ————— 8s 23ms/step - accuracy: 0.9793 - loss: 0.0691 -
val_accuracy: 0.9823 - val_loss: 0.0599
Epoch 3/15
352/352 ————— 8s 23ms/step - accuracy: 0.9848 - loss: 0.0498 -
val_accuracy: 0.9858 - val_loss: 0.0501
Epoch 4/15
352/352 ————— 8s 23ms/step - accuracy: 0.9878 - loss: 0.0377 -
val_accuracy: 0.9863 - val_loss: 0.0464
Epoch 5/15
352/352 ————— 8s 23ms/step - accuracy: 0.9910 - loss: 0.0298 -
val_accuracy: 0.9875 - val_loss: 0.0431
Epoch 6/15

```

```
352/352 ————— 8s 23ms/step - accuracy: 0.9925 - loss: 0.0232 -  
val_accuracy: 0.9872 - val_loss: 0.0421
```

Epoch 7/15

```
352/352 ————— 8s 24ms/step - accuracy: 0.9942 - loss: 0.0189 -  
val_accuracy: 0.9886 - val_loss: 0.0393
```

Epoch 8/15

```
352/352 ————— 8s 23ms/step - accuracy: 0.9951 - loss: 0.0160 -  
val_accuracy: 0.9863 - val_loss: 0.0465
```

Epoch 9/15

```
352/352 ————— 8s 23ms/step - accuracy: 0.9954 - loss: 0.0142 -  
val_accuracy: 0.9874 - val_loss: 0.0456
```

Epoch 10/15

```
352/352 ————— 8s 23ms/step - accuracy: 0.9963 - loss: 0.0117 -  
val_accuracy: 0.9895 - val_loss: 0.0408
```

Epoch 11/15

```
352/352 ————— 8s 23ms/step - accuracy: 0.9971 - loss: 0.0092 -  
val_accuracy: 0.9861 - val_loss: 0.0513
```

Epoch 12/15

```
352/352 ————— 8s 23ms/step - accuracy: 0.9972 - loss: 0.0091 -  
val_accuracy: 0.9856 - val_loss: 0.0523
```

Epoch 13/15

...

Epoch 14/15

```
352/352 ————— 8s 22ms/step - accuracy: 0.9973 - loss: 0.0082 -  
val_accuracy: 0.9893 - val_loss: 0.0467
```

Epoch 15/15

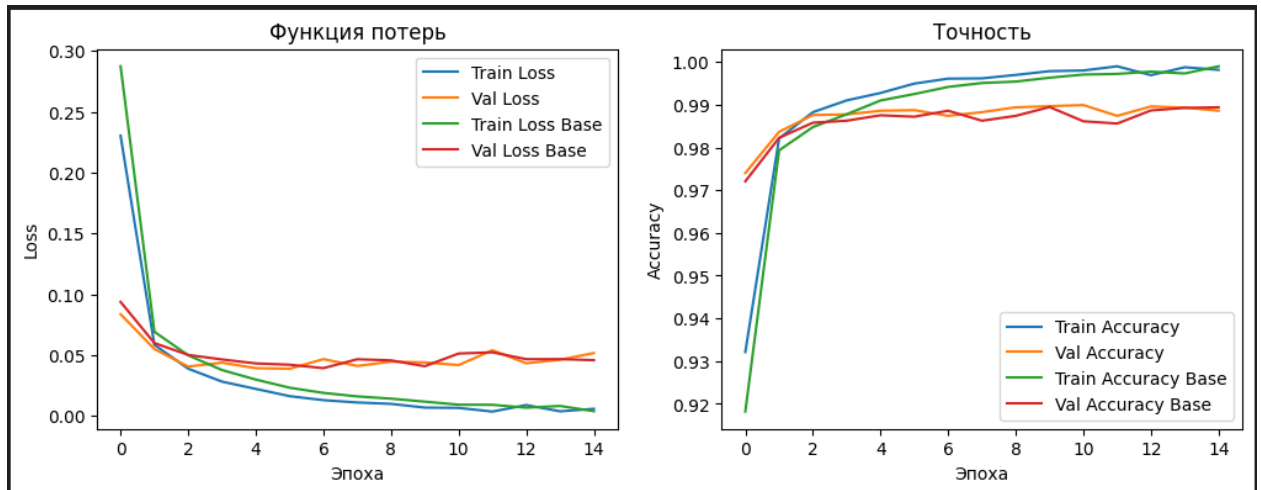
```
352/352 ————— 8s 23ms/step - accuracy: 0.9990 - loss: 0.0039 -  
val_accuracy: 0.9894 - val_loss: 0.0458
```

*Output is truncated. View as a scrollable element or open in a text editor.
Adjust cell output settings...*

Построение графиков и оценка качества модели:

Выполняется визуализация процесса обучения двух моделей: основной и базовой. Строятся сравнительные графики функции потерь и метрики точности на тренировочной и валидационной выборках. На первом графике отображается динамика изменения функции потерь, позволяющая

анализировать сходимость моделей и выявить возможное переобучение. На втором графике представлена эволюция точности классификации, демонстрирующая способность моделей к обобщению. Сравнительный анализ позволяет оценить эффективность архитектурных решений и выбрать оптимальную модель для дальнейшего использования.



```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

# График потерь
ax1.plot(history.history['loss'], label='Train Loss')
ax1.plot(history.history['val_loss'], label='Val Loss')
ax1.plot(base_history.history['loss'], label='Train Loss Base')
ax1.plot(base_history.history['val_loss'], label='Val Loss Base')
ax1.set_title('Функция потерь')
ax1.set_xlabel('Эпоха')
ax1.set_ylabel('Loss')
ax1.legend()

# График точности
ax2.plot(history.history['accuracy'], label='Train Accuracy')
ax2.plot(history.history['val_accuracy'], label='Val Accuracy')
ax2.plot(base_history.history['accuracy'], label='Train Accuracy Base')
ax2.plot(base_history.history['val_accuracy'], label='Val Accuracy Base')
ax2.set_title('Точность')
ax2.set_xlabel('Эпоха')
ax2.set_ylabel('Accuracy')
ax2.legend()

plt.show()
```

Построение графиков и оценка качества модели:

Проводится итоговая оценка качества обученных моделей на тестовом наборе данных, который не использовался в процессе обучения и валидации. Для каждой модели вычисляется значение функции потерь и точности классификации. Тестирование выполняется с параметром `verbose=0`, что отключает вывод прогресса оценки для получения компактных результатов. Сравнение точности основной и базовой моделей на независимой тестовой выборке позволяет объективно оценить их обобщающую способность и эффективность примененных архитектурных решений.

```
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f"\nТочность на тестовом наборе: {test_acc:.4f}")

test_loss, test_acc = base_model.evaluate(x_test, y_test, verbose=0)
print(f"\nТочность на тестовом наборе: {test_acc:.4f}")
```

Результат работы:

```
Точность на тестовом наборе: 0.9903
Точность на тестовом наборе: 0.9913
```

Предсказание на тестовых данных:

Выполняется тестирование обученной модели на первых 10 изображениях из тестового набора данных. Для каждого изображения модель возвращает вектор вероятностей принадлежности к каждому классу, из которого выбирается класс с максимальной вероятностью с помощью функции `argmax`. Результаты предсказания сравниваются с истинными метками, что позволяет визуально оценить качество работы классификатора. Каждое тестовое изображение отображается в градациях серого с указанием как предсказанного, так и истинного значения цифры для наглядного анализа случаев правильной и ошибочной классификации.

```
predictions = model.predict(x_test[:10])

for i in range(len(predictions)):
    predicted_label = np.argmax(predictions[i])
    true_label = np.argmax(y_test[i])
```

```

print(f"Изображение {i+1}: Предсказано = {predicted_label}, Верно = {true_label}")

# Визуализация
plt.imshow(x_test[i].squeeze(), cmap='gray')
plt.title(f"Предсказано: {predicted_label}, Истинное: {true_label}")
plt.show()

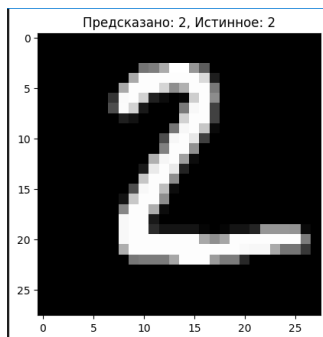
```

Результат:

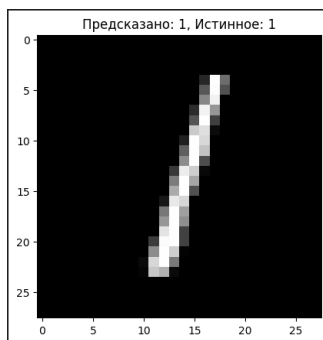
1/1 ————— 0s 32ms/step
 Изображение 1: Предсказано = 7, Верно = 7



Изображение 2: Предсказано = 2, Верно = 2



Изображение 3: Предсказано = 1, Верно = 1



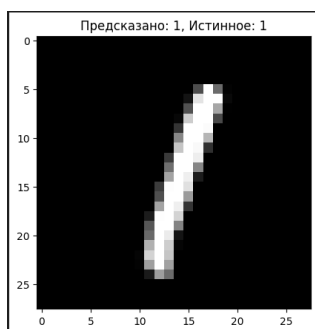
Изображение 4: Предсказано = 0, Верно = 0



Изображение 5: Предсказано = 4, Верно = 4



Изображение 6: Предсказано = 1, Верно = 1



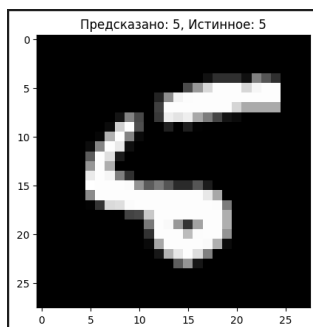
Изображение 7: Предсказано = 4, Верно = 4



Изображение 8: Предсказано = 9, Верно = 9



Изображение 9: Предсказано = 5, Верно = 5



Изображение 10: Предсказано = 9, Верно = 9



Этап 2. Построение бинарного классификатора. Обучение на малом объеме данных.

Импорт библиотек:

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np
import matplotlib.pyplot as plt
import os
import shutil
from pathlib import Path
```

Инициализация констант:

Инициализируются параметры для работы с набором данных "Cats vs Dogs". Задается размер входных изображений 150×150 пикселей, размер пакета обработки 32 образца и количество эпох обучения 15. Определяются пути к директориям с исходными и разделенными данными. Выполняется создание необходимых рабочих директорий "dataset" и "models" для хранения данных и сохранения обученных моделей соответственно. Подготовка файловой структуры обеспечивает корректную работу последующих этапов загрузки и обработки изображений.

```
# Параметры согласно варианту
IMG_HEIGHT = 150
IMG_WIDTH = 150
BATCH_SIZE = 32
NUM_EPOCHS = 15

data_dir = "dataset/dogs-vs-cats"
DATASET_PATH = f"{data_dir}-split"

# Создаем необходимые директории
Path("dataset").mkdir(exist_ok=True)
Path("models").mkdir(exist_ok=True)
```

Загрузка данных:

Реализован модуль проверки и подготовки данных для задачи бинарной классификации изображений кошек и собак. Функция `check_and_prepare_data()` выполняет проверку наличия исходных данных и при их отсутствии выводит инструкции по скачиванию датасета. При наличии данных осуществляется их организация в структурированные папки по классам с помощью функции `organize_dataset()`, которая копирует файлы в соответствующие директории "cats" и "dogs" на основе префиксов в именах файлов. Затем выполняется автоматическое разделение данных на тренировочную, валидационную и тестовую выборки в соотношении 60% 20% 20% с использованием библиотеки `splitfolders`. В случае её отсутствия

применяется альтернативное ручное разделение через функцию `manual_split_dataset()`.

```
def check_and_prepare_data():
    """Проверяет и подготавливает данные"""
    # if Path(f"{data_dir}/train").exists():
    #     print("Данные подготовлены")
    #     return True

    if not Path(data_dir).exists():
        print("Исходные данные не найдены.")
        print("Пожалуйста, выполните следующие шаги:")
        print("1. Скачайте датасет с: https://www.kaggle.com/datasets/shaunthesheep/microsoft-catsvsdogs-dataset")
        print("2. Распакуйте в папку 'datasets/dogs-vs-cats'")
        print("3. Запустите код снова")
        return False

    print("Организация данных...")
    organize_dataset()
    print("Разделение данных...")
    split_dataset()
    return True

def organize_dataset():
    """Организация данных в папки по классам"""
    src_path = f"{data_dir}/train"
    dst_path = f"{data_dir}-organized"

    Path(f"{dst_path}/cats").mkdir(parents=True, exist_ok=True)
    Path(f"{dst_path}/dogs").mkdir(parents=True, exist_ok=True)

    cnt=0
    l = len(os.listdir(src_path))
    for filename in os.listdir(src_path):
        src_file = os.path.join(src_path, filename)
        cnt+= 1
        print(f"Loaded {cnt}/{l}")
        if filename.startswith('cat') and os.path.isfile(src_file):
            shutil.copy2(src_file, f"{dst_path}/cats/{filename}")
        elif filename.startswith('dog') and os.path.isfile(src_file):
            shutil.copy2(src_file, f"{dst_path}/dogs/{filename}")
    print("Данные успешно организованы")

def split_dataset():
    """Разделение данных на train/val/test"""
    try:
        import splitfolders
        splitfolders.ratio(
```

```

        f"{data_dir}-organized",
        output=f"{data_dir}-split",
        seed=42,
        ratio=(0.6, 0.2, 0.2),
        group_prefix=None
    )
    print("Данные успешно разделены")
except ImportError:
    print("splitfolders не установлен, используем ручное разделение")
    manual_split_dataset()

def manual_split_dataset():
    """Ручное разделение данных"""
    import random
    src_path = f"{data_dir}-organized"

    for class_name in ['cats', 'dogs']:
        class_path = os.path.join(src_path, class_name)
        images = [f for f in os.listdir(class_path) if f.endswith(('.jpg',
        '.jpeg', '.png'))]
        random.shuffle(images)

        total = len(images)
        print(f"Total images for class {class_name} = {total}")
        train_end = int(total * 0.6)
        val_end = train_end + int(total * 0.2)

        for split in ['train', 'val', 'test']:
            Path(f"{data_dir}-split/{split}/{class_name}").mkdir(parents=True,
            exist_ok=True)

            for i, image in enumerate(images):
                src = os.path.join(class_path, image)
                if i < train_end:
                    dst = f"{data_dir}-split/train/{class_name}/{image}"
                elif i < val_end:
                    dst = f"{data_dir}-split/val/{class_name}/{image}"
                else:
                    dst = f"{data_dir}-split/test/{class_name}/{image}"
                shutil.copy2(src, dst)

# Проверяем и подготавливаем данные
print("Проверка данных...")
if not check_and_prepare_data():
    print("Данные не подготовлены. Завершение работы.")
    raise SystemExit("Загрузите данные и запустите код снова")

print("Создание генераторов данных...")

```

Результат работы:

Проверка данных...

Организация данных...

Loaded 1/25000

Loaded 2/25000

Loaded 3/25000

Loaded 4/25000

Loaded 5/25000

Loaded 6/25000

Loaded 7/25000

Loaded 8/25000

Loaded 9/25000

Loaded 10/25000

Loaded 11/25000

Loaded 12/25000

Loaded 13/25000

Loaded 14/25000

Loaded 15/25000

Loaded 16/25000

Loaded 17/25000

Loaded 18/25000

Loaded 19/25000

Loaded 20/25000

Loaded 21/25000

Loaded 22/25000

Loaded 23/25000

...

splitfolders не установлен, используем ручное разделение

Total images for class cats = 12500

Total images for class dogs = 12500

Создание генераторов данных...

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

Предобработка данных:

Создаются генераторы данных для обработки изображений на различных этапах работы с моделью. Для тренировочных данных применяется расширенная аугментация с целью увеличения разнообразия обучающей выборки и улучшения обобщающей способности модели. Аугментация включает случайные преобразования: вращение до 20 градусов, сдвиги по ширине и высоте до 20%, горизонтальное отражение, масштабирование до 20%, а также сдвиг на угол до 20 градусов. Все изображения нормируются в диапазон $[0, 1]$. Для валидационной и тестовой выборок используется только нормирование без аугментации, что обеспечивает корректную оценку качества модели на неизмененных данных.

```
# Аугментация для тренировочных данных. Помогает модели выделять устойчивые признаки
train_datagen = keras.preprocessing.image.ImageDataGenerator(
    rescale=1./255, # Нормирование
    rotation_range=20, # Вращение на случайный угол до 20 градусов
    width_shift_range=0.2, # Случайный сдвиг по ширине до 20%
    height_shift_range=0.2, # Случайный сдвиг по высоте до 20%
    horizontal_flip=True, # Случайное горизонтальное отражение
    zoom_range=0.2, # Случайный зум до 20%
    shear_range=0.2,
    fill_mode='nearest' # Заполнение пустых мест ближайшими пикселями
)

# Только рескейлинг для валидации и теста
val_test_datagen = keras.preprocessing.image.ImageDataGenerator(rescale=1./255)
```

Создаются генераторы данных для загрузки и обработки изображений во время обучения и оценки модели. Генераторы автоматически считывают данные из соответствующих директорий, выполняют изменение размера до 150×150 пикселей, пакетную обработку с размером батча 32 и преобразование меток в бинарный формат. Для тренировочных данных применяется перемешивание и аугментация, что способствует лучшему обобщению модели. Валидационные данные также перемешиваются, но без аугментации, а тестовые данные обрабатываются без перемешивания для сохранения

порядка при оценке. Выводится информация о соответствии классов директориям и объемах выборок.

```
# Генераторы данных. Загрузка и аугментация данных во время работы
train_generator = train_datagen.flow_from_directory(
    f"{DATASET_PATH}/train",
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary',
    shuffle=True
)

validation_generator = val_test_datagen.flow_from_directory(
    f"{DATASET_PATH}/val",
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary',
    shuffle=True
)

test_generator = val_test_datagen.flow_from_directory(
    f"{DATASET_PATH}/test",
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary',
    shuffle=False
)

print(f"Классы: {train_generator.class_indices}")
print(f"Размер тренировочной выборки: {train_generator.samples}")
print(f"Размер валидационной выборки: {validation_generator.samples}")
print(f"Размер тестовой выборки: {test_generator.samples}")
```

Результат работы:

```
Found 24371 images belonging to 2 classes.
Found 14774 images belonging to 2 classes.
Found 14754 images belonging to 2 classes.
Классы: {'cats': 0, 'dogs': 1}
Размер тренировочной выборки: 24371
Размер валидационной выборки: 14774
Размер тестовой выборки: 14754
```

Построение и обучение модели:

Создается пользовательская сверточная нейронная сеть для бинарной классификации изображений кошек и собак. Архитектура модели включает

три сверточных слоя с возрастающим количеством фильтров (32, 64, 128) и ядром 3×3 , каждый из которых следует за слоем макс-пулинга для постепенного уменьшения пространственной размерности и выделения наиболее значимых признаков. После сверточной основы выполняется преобразование в одномерный вектор с помощью слоя Flatten, а затем применяется слой Dropout с коэффициентом 0.5 для регуляризации и предотвращения переобучения. Классификатор состоит из полносвязного слоя с 512 нейронами и выходного слоя с сигмоидальной активацией для бинарной классификации. Модель компилируется с оптимизатором RMSprop и функцией потерь binary_crossentropy.

```
# Основная модель
def create_custom_model():
    model = keras.Sequential()

    model.add(layers.Conv2D(32, (3, 3), activation='relu',
input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)))
    model.add(layers.MaxPooling2D(2, 2))

    model.add(layers.Conv2D(64, (3, 3), activation='relu'))
    model.add(layers.MaxPooling2D(2, 2))

    model.add(layers.Conv2D(128, (3, 3), activation='relu'))
    model.add(layers.MaxPooling2D(2, 2))

    model.add(layers.Flatten()) # Преобразует двухмерную карту признаков в
одномерный вектор
    model.add(layers.Dropout(0.5)) # Слой, случайным образом отключающий 50%
нейронов во время обучения для борьбы с переобучением

    model.add(layers.Dense(512, activation='relu'))
    model.add(layers.Dense(1, activation='sigmoid'))
    return model

custom_model = create_custom_model()
custom_model.compile(
    optimizer=keras.optimizers.RMSprop(learning_rate=1e-4),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

custom_model.summary()
```


Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_1 (Conv2D)	(None, 72, 72, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_2 (Conv2D)	(None, 34, 34, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 17, 17, 128)	0
flatten (Flatten)	(None, 36992)	0
dropout (Dropout)	(None, 36992)	0
dense (Dense)	(None, 512)	18,940,416
dense_1 (Dense)	(None, 1)	513

Total params: 19,034,177 (72.61 MB)

Trainable params: 19,034,177 (72.61 MB)

Non-trainable params: 0 (0.00 B)

Процесс обучения длится 15 эпох с использованием генераторов тренировочных и валидационных данных. Для обеспечения наилучшего качества модели применяются callback-функции: ModelCheckpoint сохраняет веса модели с наименьшими потерями на валидационной выборке, а EarlyStopping отслеживает значение val_loss и при отсутствии улучшения в течение 10 эпох автоматически прекращает обучение, восстанавливая веса лучшей модели. По завершении обучения финальная версия модели сохраняется в файл для последующего использования.

```
custom_history = custom_model.fit(
    train_generator,
    epochs=NUM_EPOCHS,
    validation_data=validation_generator,
    callbacks=[
        keras.callbacks.ModelCheckpoint(
            filepath='models/best_custom_model.h5',
            save_best_only=True,
            monitor='val_loss',
            verbose=1
        ),
        keras.callbacks.EarlyStopping(
            monitor='val_loss',
            patience=10,
            restore_best_weights=True,
```

```

        verbose=1
    )
],
verbose=1
)

custom_model.save('models/final_custom_model.h5')

```

Epoch 1/15

762/762 ————— 0s 159ms/step - accuracy: 0.5664 - loss: 0.6766

Epoch 1: val_loss improved from None to 0.64611, saving model to models/best_custom_model.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

762/762 ————— 141s 185ms/step - accuracy: 0.6068 - loss: 0.6515 - val_accuracy: 0.6219 - val_loss: 0.6461

Epoch 2/15

762/762 ————— 0s 156ms/step - accuracy: 0.6762 - loss: 0.5986

Epoch 2: val_loss improved from 0.64611 to 0.54050, saving model to models/best_custom_model.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

762/762 ————— 140s 183ms/step - accuracy: 0.6815 - loss: 0.5939 - val_accuracy: 0.7290 - val_loss: 0.5405

Epoch 3/15

762/762 ————— 0s 160ms/step - accuracy: 0.7004 - loss: 0.5676

Epoch 3: val_loss improved from 0.54050 to 0.49691, saving model to models/best_custom_model.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

762/762 ————— 142s 186ms/step - accuracy: 0.7026 - loss: 0.5633 - val_accuracy: 0.7611 - val_loss: 0.4969

Epoch 4/15

762/762 ————— 0s 159ms/step - accuracy: 0.7180 - loss: 0.5426

Epoch 4: val_loss improved from 0.49691 to 0.48574, saving model to models/best_custom_model.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

762/762 ————— 141s 186ms/step - accuracy: 0.7220 - loss: 0.5407 - val_accuracy: 0.7613 - val_loss: 0.4857

Epoch 5/15

762/762 ————— 0s 159ms/step - accuracy: 0.7303 - loss: 0.5276

Epoch 5: val_loss improved from 0.48574 to 0.48199, saving model to models/best_custom_model.h5

```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
762/762 ----- 141s 185ms/step - accuracy: 0.7339 - loss: 0.5245 -
val_accuracy: 0.7642 - val_loss: 0.4820
Epoch 6/15
762/762 ----- 0s 156ms/step - accuracy: 0.7419 - loss: 0.5111
Epoch 6: val_loss improved from 0.48199 to 0.45000, saving model to
models/best_custom_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
762/762 ----- 139s 182ms/step - accuracy: 0.7481 - loss: 0.5058 -
val_accuracy: 0.7894 - val_loss: 0.4500
Epoch 7/15
762/762 ----- 0s 156ms/step - accuracy: 0.7578 - loss: 0.4958
Epoch 7: val_loss improved from 0.45000 to 0.43028, saving model to
models/best_custom_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
762/762 ----- 139s 182ms/step - accuracy: 0.7620 - loss: 0.4917 -
val_accuracy: 0.8017 - val_loss: 0.4303
Epoch 8/15
762/762 ----- 0s 153ms/step - accuracy: 0.7616 - loss: 0.4881
Epoch 8: val_loss did not improve from 0.43028
762/762 ----- 136s 178ms/step - accuracy: 0.7657 - loss: 0.4833 -
val_accuracy: 0.7529 - val_loss: 0.4976
Epoch 9/15
762/762 ----- 0s 152ms/step - accuracy: 0.7724 - loss: 0.4747
Epoch 9: val_loss improved from 0.43028 to 0.40336, saving model to
models/best_custom_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
762/762 ----- 136s 178ms/step - accuracy: 0.7755 - loss: 0.4711 -
val_accuracy: 0.8175 - val_loss: 0.4034
Epoch 10/15
762/762 ----- 0s 153ms/step - accuracy: 0.7825 - loss: 0.4560
Epoch 10: val_loss did not improve from 0.40336
762/762 ----- 137s 179ms/step - accuracy: 0.7795 - loss: 0.4598 -
val_accuracy: 0.8137 - val_loss: 0.4123
Epoch 11/15
762/762 ----- 0s 153ms/step - accuracy: 0.7854 - loss: 0.4534
Epoch 11: val_loss did not improve from 0.40336
762/762 ----- 136s 179ms/step - accuracy: 0.7859 - loss: 0.4527 -
val_accuracy: 0.8131 - val_loss: 0.4119
Epoch 12/15
762/762 ----- 0s 154ms/step - accuracy: 0.7867 - loss: 0.4530
Epoch 12: val_loss did not improve from 0.40336

```

```

762/762 ————— 137s 180ms/step - accuracy: 0.7895 - loss: 0.4467 -
val_accuracy: 0.8119 - val_loss: 0.4219
Epoch 13/15
762/762 ————— 0s 154ms/step - accuracy: 0.7965 - loss: 0.4358
Epoch 13: val_loss improved from 0.40336 to 0.36444, saving model to
models/best_custom_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
762/762 ————— 137s 180ms/step - accuracy: 0.7929 - loss: 0.4387 -
val_accuracy: 0.8401 - val_loss: 0.3644
Epoch 14/15
762/762 ————— 0s 152ms/step - accuracy: 0.8011 - loss: 0.4348
Epoch 14: val_loss improved from 0.36444 to 0.34759, saving model to
models/best_custom_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
762/762 ————— 136s 178ms/step - accuracy: 0.8028 - loss: 0.4292 -
val_accuracy: 0.8478 - val_loss: 0.3476
Epoch 15/15
762/762 ————— 0s 152ms/step - accuracy: 0.8069 - loss: 0.4172
Epoch 15: val_loss did not improve from 0.34759
762/762 ————— 135s 177ms/step - accuracy: 0.8044 - loss: 0.4226 -
val_accuracy: 0.8495 - val_loss: 0.3540
Restoring model weights from the end of the best epoch: 14.
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

```

Создается базовая модель сверточной нейронной сети с расширенной архитектурой для сравнения эффективности с пользовательской моделью. Модель включает четыре сверточных слоя с последовательным увеличением количества фильтров (32, 64, 128, 128) и ядрами размером 3×3, каждый из которых сопровождается слоем макс-пулинга для постепенного уменьшения размерности карт признаков. Дополнительный сверточный слой позволяет сети извлекать более сложные и абстрактные признаки из изображений. После преобразования данных в одномерный вектор применяется слой Dropout с коэффициентом 0.5 для регуляризации. Классификатор состоит из полносвязного слоя с 512 нейронами и выходного слоя с сигмоидальной

активацией для бинарной классификации. Модель компилируется с оптимизатором RMSprop и функцией потерь binary_crossentropy.

```
# Базовая модель
def create_baseline_model():
    model = keras.Sequential()

    model.add(layers.Conv2D(32, (3, 3), activation='relu',
input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)))
    model.add(layers.MaxPooling2D(2, 2))

    model.add(layers.Conv2D(64, (3, 3), activation='relu'))
    model.add(layers.MaxPooling2D(2, 2))

    model.add(layers.Conv2D(128, (3, 3), activation='relu'))
    model.add(layers.MaxPooling2D(2, 2))

    model.add(layers.Conv2D(128, (3, 3), activation='relu')) # Дополнительный
слой
    model.add(layers.MaxPooling2D(2, 2))

    model.add(layers.Flatten())
    model.add(layers.Dropout(0.5))

    model.add(layers.Dense(512, activation='relu'))
    model.add(layers.Dense(1, activation='sigmoid'))
    return model

baseline_model = create_baseline_model()
baseline_model.compile(
    optimizer=keras.optimizers.RMSprop(learning_rate=1e-4),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

baseline_model.summary()
```

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d_3 (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_4 (Conv2D)	(None, 72, 72, 64)	18,496
max_pooling2d_4 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_5 (Conv2D)	(None, 34, 34, 128)	73,856
max_pooling2d_5 (MaxPooling2D)	(None, 17, 17, 128)	0
conv2d_6 (Conv2D)	(None, 15, 15, 128)	147,584
max_pooling2d_6 (MaxPooling2D)	(None, 7, 7, 128)	0
flatten_1 (Flatten)	(None, 6272)	0
dropout_1 (Dropout)	(None, 6272)	0
dense_2 (Dense)	(None, 512)	3,211,776
dense_3 (Dense)	(None, 1)	513

Total params: 3,453,121 (13.17 MB)

Trainable params: 3,453,121 (13.17 MB)

Осуществляется обучение базовой модели сверточной нейронной сети с использованием тех же параметров, что и для пользовательской модели. Процесс обучения длится 15 эпох с применением генераторов данных для тренировочной и валидационной выборок. Для оптимизации процесса обучения и предотвращения переобучения используются callback-функции: ModelCheckpoint сохраняет весовые коэффициенты модели с наименьшим значением потерь на валидационной выборке, а EarlyStopping отслеживает динамику val_loss и автоматически прекращает обучение при отсутствии улучшений в течение 10 эпох, восстанавливая параметры лучшей версии модели. После завершения обучения финальная модель сохраняется в файл для последующего анализа и использования.

```
baseline_history = baseline_model.fit(
    train_generator,
    epochs=NUM_EPOCHS,
    validation_data=validation_generator,
    callbacks=[
        keras.callbacks.ModelCheckpoint(
            filepath='models/best_baseline_model.h5',
            save_best_only=True,
            monitor='val_loss',
            verbose=1
        ),
        keras.callbacks.EarlyStopping(
            monitor='val_loss',
```

```

        patience=10,
        restore_best_weights=True,
        verbose=1
    )
],
verbose=1
)

baseline_model.save('models/final_baseline_model.h5')

```

```

Epoch 1/15
762/762 ————— 0s 135ms/step - accuracy: 0.5490 - loss: 0.6821
Epoch 1: val_loss improved from None to 0.61483, saving model to
models/best_baseline_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
762/762 ————— 121s 158ms/step - accuracy: 0.5787 - loss: 0.6695 -
val_accuracy: 0.6580 - val_loss: 0.6148
Epoch 2/15
762/762 ————— 0s 137ms/step - accuracy: 0.6511 - loss: 0.6195
Epoch 2: val_loss improved from 0.61483 to 0.55878, saving model to
models/best_baseline_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
762/762 ————— 122s 160ms/step - accuracy: 0.6612 - loss: 0.6100 -
val_accuracy: 0.7112 - val_loss: 0.5588
Epoch 3/15
762/762 ————— 0s 136ms/step - accuracy: 0.6894 - loss: 0.5795
Epoch 3: val_loss improved from 0.55878 to 0.51492, saving model to
models/best_baseline_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
762/762 ————— 121s 159ms/step - accuracy: 0.6946 - loss: 0.5741 -
val_accuracy: 0.7473 - val_loss: 0.5149
Epoch 4/15
762/762 ————— 0s 136ms/step - accuracy: 0.7109 - loss: 0.5522
Epoch 4: val_loss improved from 0.51492 to 0.50508, saving model to
models/best_baseline_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
762/762 ————— 121s 158ms/step - accuracy: 0.7131 - loss: 0.5506 -
val_accuracy: 0.7477 - val_loss: 0.5051
Epoch 5/15
762/762 ————— 0s 137ms/step - accuracy: 0.7324 - loss: 0.5302

```

Epoch 5: val_loss improved from 0.50508 to 0.45935, saving model to models/best_baseline_model.h5
 WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

762/762 ————— 122s 160ms/step - accuracy: 0.7364 - loss: 0.5265 - val_accuracy: 0.8012 - val_loss: 0.4594

Epoch 6/15

762/762 ————— 0s 136ms/step - accuracy: 0.7481 - loss: 0.5070

Epoch 6: val_loss improved from 0.45935 to 0.44255, saving model to models/best_baseline_model.h5
 WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

762/762 ————— 121s 159ms/step - accuracy: 0.7516 - loss: 0.5017 - val_accuracy: 0.7949 - val_loss: 0.4426

Epoch 7/15

762/762 ————— 0s 136ms/step - accuracy: 0.7656 - loss: 0.4904

Epoch 7: val_loss improved from 0.44255 to 0.41887, saving model to models/best_baseline_model.h5
 WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

762/762 ————— 122s 160ms/step - accuracy: 0.7662 - loss: 0.4882 - val_accuracy: 0.8096 - val_loss: 0.4189

Epoch 8/15

762/762 ————— 0s 136ms/step - accuracy: 0.7669 - loss: 0.4780

Epoch 8: val_loss improved from 0.41887 to 0.39855, saving model to models/best_baseline_model.h5
 WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

762/762 ————— 121s 159ms/step - accuracy: 0.7714 - loss: 0.4739 - val_accuracy: 0.8193 - val_loss: 0.3985

Epoch 9/15

762/762 ————— 0s 135ms/step - accuracy: 0.7865 - loss: 0.4563

Epoch 9: val_loss did not improve from 0.39855

762/762 ————— 120s 157ms/step - accuracy: 0.7863 - loss: 0.4551 - val_accuracy: 0.7980 - val_loss: 0.4270

Epoch 10/15

762/762 ————— 0s 136ms/step - accuracy: 0.7864 - loss: 0.4548

Epoch 10: val_loss improved from 0.39855 to 0.38153, saving model to models/best_baseline_model.h5
 WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

762/762 ————— 121s 158ms/step - accuracy: 0.7845 - loss: 0.4535 - val_accuracy: 0.8314 - val_loss: 0.3815


```

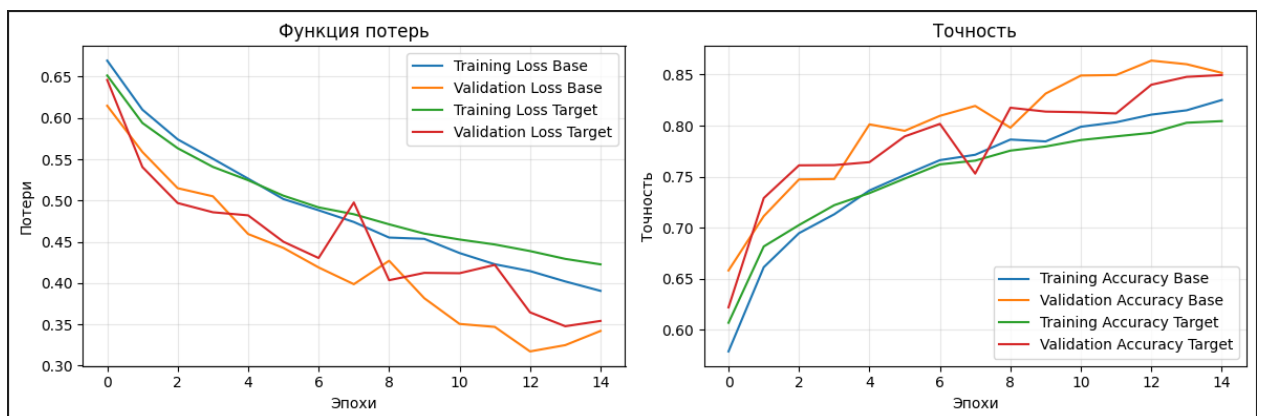
Epoch 11/15
762/762 ————— 0s 142ms/step - accuracy: 0.7990 - loss: 0.4344
Epoch 11: val_loss improved from 0.38153 to 0.35040, saving model to
models/best_baseline_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
762/762 ————— 126s 166ms/step - accuracy: 0.7989 - loss: 0.4363 -
val_accuracy: 0.8491 - val_loss: 0.3504
Epoch 12/15
762/762 ————— 0s 147ms/step - accuracy: 0.8021 - loss: 0.4226
Epoch 12: val_loss improved from 0.35040 to 0.34678, saving model to
models/best_baseline_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
762/762 ————— 131s 171ms/step - accuracy: 0.8033 - loss: 0.4227 -
val_accuracy: 0.8496 - val_loss: 0.3468
Epoch 13/15
762/762 ————— 0s 152ms/step - accuracy: 0.8054 - loss: 0.4198
Epoch 13: val_loss improved from 0.34678 to 0.31708, saving model to
models/best_baseline_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
762/762 ————— 135s 177ms/step - accuracy: 0.8108 - loss: 0.4145 -
val_accuracy: 0.8637 - val_loss: 0.3171
Epoch 14/15
762/762 ————— 0s 145ms/step - accuracy: 0.8130 - loss: 0.4049
Epoch 14: val_loss did not improve from 0.31708
762/762 ————— 128s 168ms/step - accuracy: 0.8150 - loss: 0.4019 -
val_accuracy: 0.8602 - val_loss: 0.3247
Epoch 15/15
762/762 ————— 0s 140ms/step - accuracy: 0.8247 - loss: 0.3913
Epoch 15: val_loss did not improve from 0.31708
762/762 ————— 125s 164ms/step - accuracy: 0.8251 - loss: 0.3905 -
val_accuracy: 0.8517 - val_loss: 0.3419
Restoring model weights from the end of the best epoch: 13.
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

```

Графики и оценка качества:

Выполняется сравнительный анализ процесса обучения базовой и пользовательской моделей путем визуализации динамики функции потерь и

метрики точности на тренировочной и валидационной выборках. Строятся два графика: первый отображает изменение функции потерь в зависимости от эпох обучения, второй показывает эволюцию точности классификации. На каждом графике представлены кривые для обеих моделей, что позволяет провести сравнительную оценку их сходимости, способности к обобщению и выявить возможное переобучение. Использование сетки повышает читаемость графиков, а компактное расположение subplot'ов обеспечивает наглядность сравнения.



```
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(baseline_history.history['loss'], label='Training Loss Base')
plt.plot(baseline_history.history['val_loss'], label='Validation Loss Base')
plt.plot(custom_history.history['loss'], label='Training Loss Target')
plt.plot(custom_history.history['val_loss'], label='Validation Loss Target')
plt.title('Функция потерь')
plt.xlabel('Эпохи')
plt.ylabel('Потери')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.plot(baseline_history.history['accuracy'], label='Training Accuracy Base')
plt.plot(baseline_history.history['val_accuracy'], label='Validation Accuracy Base')
plt.plot(custom_history.history['accuracy'], label='Training Accuracy Target')
plt.plot(custom_history.history['val_accuracy'], label='Validation Accuracy Target')
plt.title('Точность')
plt.xlabel('Эпохи')
plt.ylabel('Точность')
plt.legend()
```

```
plt.grid(True, alpha=0.3)
plt.tight_layout()

plt.show()
```

Проводится итоговая оценка качества обученных моделей на независимом тестовом наборе данных, который не использовался в процессе обучения и валидации. Для каждой модели вычисляются значения функции потерь и точности классификации. Тестирование выполняется с параметром `verbose=0`, что отключает вывод прогресса оценки для получения компактных результатов. Сравнение точности базовой и основной моделей на тестовой выборке позволяет объективно оценить их обобщающую способность и эффективность архитектурных решений, выявив наиболее подходящую модель для решения задачи бинарной классификации изображений кошек и собак.

```
Базовая модель - Тестовая точность: 0.8630
Основная модель - Тестовая точность: 0.8479
```

Предсказание на тестовых данных:

Выполняется визуальная демонстрация работы обученных моделей на примерах из тестовой выборки. Из тестового генератора извлекается пакет изображений и соответствующих меток. Для каждого из первых 10 изображений в пакете модели выполняют предсказание, определяя вероятность принадлежности к классу "собака". На основе порогового значения 0.5 вероятности преобразуются в текстовые метки классов. Строится компактная сетка изображений, где для каждого примера отображается исходное изображение, истинная метка, а также предсказания базовой и основной моделей. Это позволяет визуально оценить качество классификации и сравнить работу разных архитектур на конкретных примерах.

```
test_images, test_labels = next(test_generator)
num_demo = min(10, BATCH_SIZE)
```

```

plt.figure(figsize=(15, 3))
for i in range(num_demo):
    image = test_images[i]
    true_label = "кошка" if test_labels[i] == 0 else "собака"

    baseline_pred = baseline_model.predict(np.expand_dims(image, axis=0),
verbose=0)[0][0]
    custom_pred = custom_model.predict(np.expand_dims(image, axis=0),
verbose=0)[0][0]

    baseline_class = "кошка" if baseline_pred < 0.5 else "собака"
    custom_class = "кошка" if custom_pred < 0.5 else "собака"

    plt.subplot(1, num_demo, i + 1)
    plt.imshow(image)
    plt.title(f'Истина: {true_label}\nБазовая: {baseline_class}\nОсновная:
{custom_class}')
    plt.axis('off')

plt.tight_layout()
plt.show()

```



Этап 3. Использование предварительно обученной сети.

Импорт библиотек:

```

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.applications import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input
import numpy as np
import matplotlib.pyplot as plt
import os
from pathlib import Path
import ssl
import urllib

```

Инициализация констант:

Инициализируются параметры для третьего этапа работы с использованием предобученной модели. Задается размер входных изображений 150×150 пикселей, размер пакета обработки 32 образца и количество эпох обучения 40. Определяется путь к предварительно разделенным данным. Выполняется проверка существования подготовленных данных, которые должны были быть созданы на втором этапе работы. При отсутствии необходимых данных работа прерывается с соответствующим сообщением, что обеспечивает корректность выполнения последующих операций с данными.

```
IMG_HEIGHT = 150
IMG_WIDTH = 150
BATCH_SIZE = 32
NUM_EPOCHS = 40

DATASET_PATH = "dataset/dogs-vs-cats-split"
if not Path(DATASET_PATH).exists():
    print("Данные не найдены. Завершение работы.")
    raise SystemExit("Данные должны были быть предобработаны во втором этапе")
```

Предобработка данных:

Создаются генераторы данных для извлечения признаков с использованием предобученной модели. В отличие от предыдущего этапа, аугментация данных не применяется, поскольку целью является получение стабильных признаков из сверточной основы. Для обработки изображений используется функция `grprocess_input`, которая обеспечивает соответствующую предобработку данных для совместимости с предобученной моделью VGG16. Генераторы загружают изображения из соответствующих директорий, изменяют их размер до 150×150 пикселей, организуют пакетную обработку и преобразуют метки в бинарный формат. Тренировочные и валидационные данные перемешиваются, тогда как тестовые сохраняют исходный порядок для последующей оценки.

```
# Создание генераторов данных без аугментации (для feature extraction)
print("Создание генераторов данных...")
```

```

datagen = keras.preprocessing.image.ImageDataGenerator(
    preprocessing_function=preprocess_input
)

train_generator = datagen.flow_from_directory(
    f"{DATASET_PATH}/train",
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary',
    shuffle=True
)

validation_generator = datagen.flow_from_directory(
    f"{DATASET_PATH}/val",
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary',
    shuffle=True
)

test_generator = datagen.flow_from_directory(
    f"{DATASET_PATH}/test",
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary',
    shuffle=False
)

print(f"Размер тренировочной выборки: {train_generator.samples}")
print(f"Размер валидационной выборки: {validation_generator.samples}")
print(f"Размер тестовой выборки: {test_generator.samples}")

```

```

Создание генераторов данных...
Found 24371 images belonging to 2 classes.
Found 14774 images belonging to 2 classes.
Found 14754 images belonging to 2 classes.
Размер тренировочной выборки: 24371
Размер валидационной выборки: 14774
Размер тестовой выборки: 14754

```

Загрузка предобученной модели и обучение:

Загружается предобученная модель VGG16 для использования в качестве экстрактора признаков. Модель загружается с весовыми коэффициентами, полученными в результате обучения на наборе данных ImageNet, при этом классификационная часть (верхние слои) исключается из архитектуры. Все

слои сверточной основы замораживаются, что предотвращает их обновление в процессе дальнейшего обучения и позволяет использовать уже сформированные веса для извлечения универсальных признаков из изображений.

```
print("\nЗагрузка предобученной модели VGG16...")

vgg_base = VGG16(
    weights='/Users/krukrukruzhka/github_projects/ML_magister/vgg16_weights_notop.h5',
    include_top=False,
    input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)
)

# Заморозка сверточной основы
vgg_base.trainable = False

print("Архитектура сверточной основы VGG16:")
vgg_base.summary()
```

Загрузка предобученной модели VGG16...
Архитектура сверточной основы VGG16:

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 150, 150, 3)	0
block1_conv1 (Conv2D)	(None, 150, 150, 64)	1,792
block1_conv2 (Conv2D)	(None, 150, 150, 64)	36,928
block1_pool (MaxPooling2D)	(None, 75, 75, 64)	0
block2_conv1 (Conv2D)	(None, 75, 75, 128)	73,856
block2_conv2 (Conv2D)	(None, 75, 75, 128)	147,584
block2_pool (MaxPooling2D)	(None, 37, 37, 128)	0
block3_conv1 (Conv2D)	(None, 37, 37, 256)	295,168
block3_conv2 (Conv2D)	(None, 37, 37, 256)	590,080
block3_conv3 (Conv2D)	(None, 37, 37, 256)	590,080
block3_pool (MaxPooling2D)	(None, 18, 18, 256)	0
block4_conv1 (Conv2D)	(None, 18, 18, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 18, 18, 512)	2,359,808

block4_conv3 (Conv2D)	(None, 18, 18, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 9, 9, 512)	0
block5_conv1 (Conv2D)	(None, 9, 9, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 9, 9, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 9, 9, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 4, 4, 512)	0

Создание основной модели:

Создается классификатор для использования совместно с предобученной сверточной основой VGG16. Архитектура классификатора соответствует варианту из второго этапа работы и включает слой Flatten для преобразования многомерных карт признаков в одномерный вектор, слой Dropout с коэффициентом 0.5 для регуляризации и предотвращения переобучения, полносвязный слой с 512 нейронами и активацией ReLU, а также выходной слой с сигмоидальной активацией для бинарной классификации. Модель компилируется с оптимизатором RMSprop и функцией потерь binary_crossentropy, что обеспечивает согласованность подходов при сравнении результатов различных этапов работы.

```
# Основная модель
def create_classifier():
    """Классификатор из этапа 2"""
    model = keras.Sequential()
    model.add(layers.Flatten(input_shape=(4, 4, 512)))
    model.add(layers.Dropout(0.5))
    model.add(layers.Dense(512, activation='relu')) # 512 нейронов как в этапе 2
    model.add(layers.Dense(1, activation='sigmoid'))
    return model

classifier = create_classifier()

classifier.summary()

classifier.compile(
    optimizer=keras.optimizers.RMSprop(learning_rate=1e-4),
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```


Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 8192)	0
dropout (Dropout)	(None, 8192)	0
dense (Dense)	(None, 512)	4,194,816
dense_1 (Dense)	(None, 1)	513

Total params: 4,195,329 (16.00 MB)

Trainable params: 4,195,329 (16.00 MB)

Non-trainable params: 0 (0.00 B)

Извлечение признаков:

Реализуется процедура извлечения признаков из изображений с использованием предобученной сверточной основы VGG16. Функция `extract_features` последовательно обрабатывает пакеты данных из генератора, пропуская их через замороженную модель VGG16 и сохраняя полученные карты признаков размерностью $4 \times 4 \times 512$. Процесс выполняется для тренировочной, валидационной и тестовой выборок, при этом сохраняются как извлеченные признаки, так и соответствующие метки классов. Такой подход позволяет один раз вычислить признаки для всего набора данных и затем эффективно обучать классификатор, значительно ускоряя процесс обучения по сравнению с end-to-end подходом.

```
def extract_features(generator, model, sample_count):
    """Извлечение признаков с помощью VGG16"""
    features = np.zeros(shape=(sample_count, 4, 4, 512))
    labels = np.zeros(shape=(sample_count))

    i = 0
    for inputs_batch, labels_batch in generator:
        features_batch = model.predict(inputs_batch, verbose=1)
        features[i * BATCH_SIZE : (i + 1) * BATCH_SIZE] = features_batch
        labels[i * BATCH_SIZE : (i + 1) * BATCH_SIZE] = labels_batch
        i += 1
        if i * BATCH_SIZE >= sample_count:
            break
```

```

    return features, labels

print("\nИзвлечение признаков из тренировочных данных...")
train_features, train_labels = extract_features(train_generator, vgg_base,
train_generator.samples)

print("Извлечение признаков из валидационных данных...")
validation_features, validation_labels = extract_features(validation_generator,
vgg_base, validation_generator.samples)

print("Извлечение признаков из тестовых данных...")
test_features, test_labels = extract_features(test_generator, vgg_base,
test_generator.samples)

print(f"Размеры извлеченных признаков:")
print(f"Тренировочные: {train_features.shape}")
print(f"Валидационные: {validation_features.shape}")
print(f"Тестовые: {test_features.shape}")

```

Извлечение признаков из тренировочных данных...

```

1/1 ————— 1s 726ms/step
1/1 ————— 1s 645ms/step
1/1 ————— 1s 793ms/step
1/1 ————— 1s 656ms/step
1/1 ————— 1s 650ms/step
1/1 ————— 1s 704ms/step
1/1 ————— 1s 664ms/step
1/1 ————— 1s 703ms/step
1/1 ————— 1s 656ms/step
1/1 ————— 1s 650ms/step
1/1 ————— 1s 803ms/step
1/1 ————— 1s 641ms/step
1/1 ————— 1s 642ms/step
1/1 ————— 1s 699ms/step
1/1 ————— 1s 655ms/step
1/1 ————— 1s 644ms/step
1/1 ————— 1s 707ms/step
1/1 ————— 1s 701ms/step
1/1 ————— 1s 648ms/step
1/1 ————— 1s 802ms/step

```

```
1/1 ————— 1s 674ms/step
1/1 ————— 1s 710ms/step
1/1 ————— 1s 642ms/step
...
```

Размеры извлеченных признаков:

Тренировочные: (24371, 4, 4, 512)

Валидационные: (14774, 4, 4, 512)

Тестовые: (14754, 4, 4, 512)

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)..

Обучение основной модели:

Выполняется обучение классификатора на извлеченных признаках с использованием предобученной сверточной основы VGG16. Процесс обучения длится 40 эпох с размером пакета 32 образца. Для оптимизации обучения применяются callback-функции: ModelCheckpoint сохраняет веса модели с наименьшими потерями на валидационной выборке, а EarlyStopping отслеживает значение val_loss и автоматически прекращает обучение при отсутствии улучшений в течение 10 эпох, восстанавливая параметры лучшей версии модели. Обучение проводится непосредственно на извлеченных признаках, что значительно ускоряет процесс по сравнению с end-to-end обучением и позволяет достичь высокой точности даже при небольшом объеме данных.

```
history = classifier.fit(
    train_features, train_labels,
    epochs=NUM_EPOCHS,
    batch_size=BATCH_SIZE,
    validation_data=(validation_features, validation_labels),
    callbacks=[
        keras.callbacks.ModelCheckpoint(
            filepath='models/best_vgg_classifier.h5',
            save_best_only=True,
            monitor='val_loss',
            verbose=1
        ),
        keras.callbacks.EarlyStopping(
            monitor='val_loss',
```

```

        patience=10,
        restore_best_weights=True,
        verbose=1
    )
],
verbose=1
)

# # Загрузка лучшей модели
# classifier = keras.models.load_model('models/best_vgg_classifier.h5')

```

Epoch 1/40

758/762 ————— 0s 7ms/step - accuracy: 0.9186 - loss: 1.5778

Epoch 1: val_loss improved from None to 0.28699, saving model to models/best_vgg_classifier.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

762/762 ————— 7s 9ms/step - accuracy: 0.9410 - loss: 1.0815 - val_accuracy: 0.9762 - val_loss: 0.2870

Epoch 2/40

761/762 ————— 0s 8ms/step - accuracy: 0.9623 - loss: 0.5263

Epoch 2: val_loss improved from 0.28699 to 0.07286, saving model to models/best_vgg_classifier.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

762/762 ————— 7s 9ms/step - accuracy: 0.9631 - loss: 0.4782 - val_accuracy: 0.9867 - val_loss: 0.0729

Epoch 3/40

762/762 ————— 0s 8ms/step - accuracy: 0.9691 - loss: 0.2670

Epoch 3: val_loss improved from 0.07286 to 0.03742, saving model to models/best_vgg_classifier.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

762/762 ————— 7s 9ms/step - accuracy: 0.9684 - loss: 0.2705 - val_accuracy: 0.9895 - val_loss: 0.0374

Epoch 4/40

760/762 ————— 0s 7ms/step - accuracy: 0.9749 - loss: 0.1696

Epoch 4: val_loss improved from 0.03742 to 0.03046, saving model to models/best_vgg_classifier.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

762/762 ————— 7s 9ms/step - accuracy: 0.9757 - loss: 0.1645 - val_accuracy: 0.9903 - val_loss: 0.0305

Epoch 5/40

758/762 ————— 0s 8ms/step - accuracy: 0.9780 - loss: 0.1230

Epoch 5: val_loss improved from 0.03046 to 0.01718, saving model to models/best_vgg_classifier.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

762/762 ————— 7s 9ms/step - accuracy: 0.9776 - loss: 0.1353 - val_accuracy: 0.9948 - val_loss: 0.0172

Epoch 6/40

760/762 ————— 0s 8ms/step - accuracy: 0.9848 - loss: 0.0836

Epoch 6: val_loss improved from 0.01718 to 0.01253, saving model to models/best_vgg_classifier.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

762/762 ————— 7s 9ms/step - accuracy: 0.9821 - loss: 0.1003 - val_accuracy: 0.9968 - val_loss: 0.0125

Epoch 7/40

757/762 ————— 0s 8ms/step - accuracy: 0.9844 - loss: 0.0888

Epoch 7: val_loss did not improve from 0.01253

762/762 ————— 7s 9ms/step - accuracy: 0.9836 - loss: 0.0914 - val_accuracy: 0.9962 - val_loss: 0.0138

Epoch 8/40

757/762 ————— 0s 8ms/step - accuracy: 0.9850 - loss: 0.0784

Epoch 8: val_loss did not improve from 0.01253

762/762 ————— 7s 9ms/step - accuracy: 0.9860 - loss: 0.0737 -
val_accuracy: 0.9958 - val_loss: 0.0137

Epoch 9/40

756/762 ————— 0s 8ms/step - accuracy: 0.9867 - loss: 0.0656

Epoch 9: val_loss improved from 0.01253 to 0.01091, saving model to
models/best_vgg_classifier.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save\_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my\_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

762/762 ————— 7s 9ms/step - accuracy: 0.9867 - loss: 0.0657 -
val_accuracy: 0.9965 - val_loss: 0.0109

Epoch 10/40

760/762 ————— 0s 8ms/step - accuracy: 0.9891 - loss: 0.0465

Epoch 10: val_loss improved from 0.01091 to 0.00921, saving model to
models/best_vgg_classifier.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save\_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my\_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

762/762 ————— 7s 9ms/step - accuracy: 0.9886 - loss: 0.0538 -
val_accuracy: 0.9977 - val_loss: 0.0092

Epoch 11/40

755/762 ————— 0s 7ms/step - accuracy: 0.9900 - loss: 0.0478

Epoch 11: val_loss improved from 0.00921 to 0.00783, saving model to
models/best_vgg_classifier.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save\_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my\_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

762/762 ————— 7s 9ms/step - accuracy: 0.9897 - loss: 0.0522 -
val_accuracy: 0.9985 - val_loss: 0.0078

Epoch 12/40

760/762 ————— 0s 8ms/step - accuracy: 0.9905 - loss: 0.0477

Epoch 12: val_loss improved from 0.00783 to 0.00719, saving model to
models/best_vgg_classifier.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save\_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.

```
`model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
```

```
762/762 ————— 7s 9ms/step - accuracy: 0.9897 - loss: 0.0532 -  
val_accuracy: 0.9986 - val_loss: 0.0072
```

Epoch 13/40

```
761/762 ————— 0s 8ms/step - accuracy: 0.9923 - loss: 0.0369
```

Epoch 13: val_loss did not improve from 0.00719

```
762/762 ————— 7s 9ms/step - accuracy: 0.9919 - loss: 0.0419 -  
val_accuracy: 0.9986 - val_loss: 0.0081
```

Epoch 14/40

```
756/762 ————— 0s 8ms/step - accuracy: 0.9924 - loss: 0.0361
```

Epoch 14: val_loss improved from 0.00719 to 0.00631, saving model to models/best_vgg_classifier.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g.

```
`model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
```

```
762/762 ————— 7s 9ms/step - accuracy: 0.9921 - loss: 0.0375 -  
val_accuracy: 0.9987 - val_loss: 0.0063
```

Epoch 15/40

```
760/762 ————— 0s 7ms/step - accuracy: 0.9934 - loss: 0.0280
```

Epoch 15: val_loss did not improve from 0.00631

```
762/762 ————— 7s 9ms/step - accuracy: 0.9925 - loss: 0.0294 -  
val_accuracy: 0.9987 - val_loss: 0.0069
```

Epoch 16/40

```
761/762 ————— 0s 8ms/step - accuracy: 0.9920 - loss: 0.0372
```

Epoch 16: val_loss improved from 0.00631 to 0.00624, saving model to models/best_vgg_classifier.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g.

```
`model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
```

```
762/762 ————— 7s 9ms/step - accuracy: 0.9920 - loss: 0.0358 -  
val_accuracy: 0.9988 - val_loss: 0.0062
```

Epoch 17/40

```
759/762 ————— 0s 7ms/step - accuracy: 0.9933 - loss: 0.0335
```

Epoch 17: val_loss did not improve from 0.00624

762/762 ————— 7s 9ms/step - accuracy: 0.9932 - loss: 0.0333 -
val_accuracy: 0.9989 - val_loss: 0.0066

Epoch 18/40

758/762 ————— 0s 8ms/step - accuracy: 0.9950 - loss: 0.0247

Epoch 18: val_loss did not improve from 0.00624

762/762 ————— 7s 9ms/step - accuracy: 0.9939 - loss: 0.0281 -
val_accuracy: 0.9988 - val_loss: 0.0067

Epoch 19/40

760/762 ————— 0s 8ms/step - accuracy: 0.9954 - loss: 0.0227

Epoch 19: val_loss did not improve from 0.00624

762/762 ————— 7s 9ms/step - accuracy: 0.9953 - loss: 0.0236 -
val_accuracy: 0.9988 - val_loss: 0.0070

Epoch 20/40

757/762 ————— 0s 8ms/step - accuracy: 0.9945 - loss: 0.0201

Epoch 20: val_loss did not improve from 0.00624

762/762 ————— 8s 10ms/step - accuracy: 0.9948 - loss: 0.0177 -
val_accuracy: 0.9988 - val_loss: 0.0066

Epoch 21/40

755/762 ————— 0s 8ms/step - accuracy: 0.9957 - loss: 0.0213

Epoch 21: val_loss did not improve from 0.00624

762/762 ————— 7s 9ms/step - accuracy: 0.9958 - loss: 0.0195 -
val_accuracy: 0.9990 - val_loss: 0.0063

Epoch 22/40

758/762 ————— 0s 7ms/step - accuracy: 0.9934 - loss: 0.0276

Epoch 22: val_loss did not improve from 0.00624

762/762 ————— 7s 9ms/step - accuracy: 0.9945 - loss: 0.0248 -
val_accuracy: 0.9988 - val_loss: 0.0074

...

Epoch 26: val_loss did not improve from 0.00624

762/762 ————— 7s 9ms/step - accuracy: 0.9966 - loss: 0.0163 -
val_accuracy: 0.9987 - val_loss: 0.0070

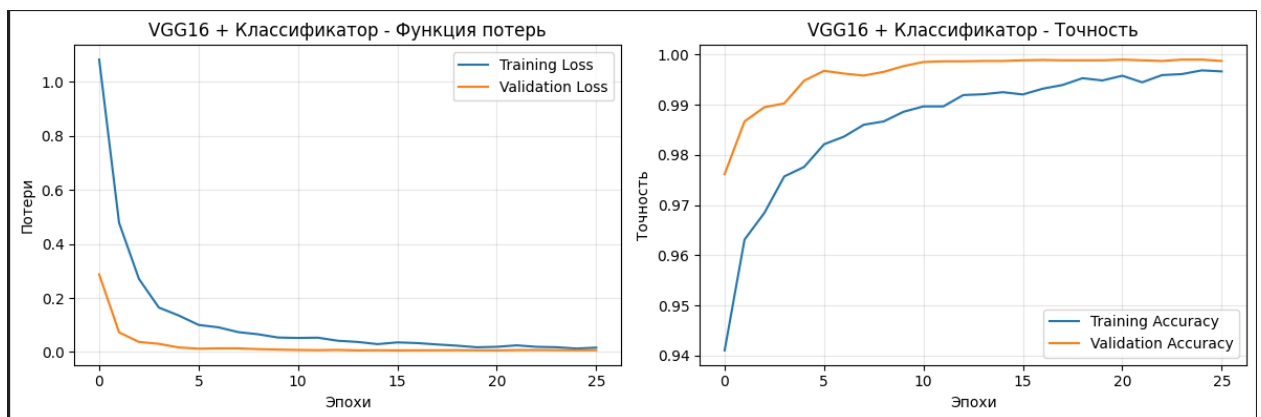
Epoch 26: early stopping

Restoring model weights from the end of the best epoch: 16.

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)..

Графики и оценка качества:

Выполняется визуализация процесса обучения классификатора на признаках, извлеченных с помощью предобученной модели VGG16. Строятся два графика: первый отображает динамику функции потерь на тренировочной и валидационной выборках, второй показывает эволюцию метрики точности в процессе обучения. Графики позволяют проанализировать сходимость модели, выявить возможное переобучение и оценить эффективность подхода выделения признаков. Использование сетки повышает читаемость графиков, а компактное расположение обеспечивает наглядность анализа процесса обучения классификатора в рамках transfer learning.



```
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('VGG16 + Классификатор - Функция потерь')
plt.xlabel('Эпохи')
plt.ylabel('Потери')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('VGG16 + Классификатор - Точность')
plt.xlabel('Эпохи')
plt.ylabel('Точность')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
```

```
plt.show()
```

Проводится итоговая оценка качества обученного классификатора на независимом тестовом наборе данных. Модель оценивается на признаках, извлеченных из тестовых изображений с помощью предобученной сверточной основы VGG16. Вычисляются значения функции потерь и точности классификации, что позволяет объективно оценить эффективность подхода transfer learning с выделением признаков. Тестирование выполняется с параметром `verbose=1`, обеспечивающим вывод прогресса оценки. Полученные метрики демонстрируют обобщающую способность модели и позволяют сравнить результаты с предыдущими этапами работы.

```
test_loss, test_accuracy = classifier.evaluate(test_features, test_labels,
verbose=1)
print(f"Тестовые потери: {test_loss:.4f}")
print(f"Тестовая точность: {test_accuracy:.4f}")
```

```
462/462 ————— 1s 2ms/step - accuracy: 0.9987 - loss: 0.0064
Тестовые потери: 0.0064
Тестовая точность: 0.9987
```

Предсказание на тестовых данных:

Реализуется функция для предсказания класса отдельного изображения с использованием подхода transfer learning. Функция выполняет полный цикл обработки: загрузку изображения с изменением размера до 150×150 пикселей, преобразование в массив numpy, предобработку с помощью функции `preprocess_input` для совместимости с VGG16, извлечение признаков через замороженную сверточную основу VGG16 и окончательное предсказание с помощью обученного классификатора. Результат представляется в виде текстовой метки класса ("кошка" или "собака") и численного значения уверенности предсказания. Визуализация включает отображение исходного изображения с аннотацией, содержащей предсказание и уровень уверенности модели.

```
def predict_with_vgg(model, vgg_base, image_path):
    """Предсказание для одного изображения"""
    # Загрузка и подготовка изображения
    img = keras.preprocessing.image.load_img(image_path, target_size=(IMG_HEIGHT,
IMG_WIDTH))
    img_array = keras.preprocessing.image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0)
    img_array = preprocess_input(img_array)

    # Извлечение признаков
    features = vgg_base.predict(img_array, verbose=0)

    # Предсказание
    prediction = model.predict(features, verbose=0)[0][0]
    class_name = "кошка" if prediction < 0.5 else "собака"
    confidence = (1 - prediction) if prediction < 0.5 else prediction

    # Визуализация
    plt.figure(figsize=(6, 6))
    plt.imshow(img)
    plt.title(f'VGG16 + Классификатор\nПредсказание: {class_name}\nУверенность:
{confidence:.2%}')
    plt.axis('off')
    plt.show()

    return class_name, confidence
```

Выполняется демонстрация работы модели VGG16 с дообученным классификатором на примерах из тестовой выборки. Из тестового генератора извлекается пакет изображений и соответствующих меток. Для каждого из первых 10 изображений в пакете выполняется полный цикл предсказания: изображение пропускается через сверточную основу VGG16 для извлечения признаков, затем классификатор определяет вероятность принадлежности к классу "собака". На основе порогового значения 0.5 вероятности преобразуются в текстовые метки классов и вычисляется уровень уверенности предсказания. Строится компактная сетка изображений с аннотациями, содержащими истинные метки, предсказания модели и значения уверенности.

```
test_images, test_true_labels = next(test_generator)
num_demo = min(10, BATCH_SIZE)

plt.figure(figsize=(15, 3))
for i in range(num_demo):
```

```

img_array = test_images[i]
img_array = np.expand_dims(img_array, axis=0)

features = vgg_base.predict(img_array, verbose=0)
prediction = classifier.predict(features, verbose=0)[0][0]

true_label = "кошка" if test_true_labels[i] == 0 else "собака"
predicted_class = "кошка" if prediction < 0.5 else "собака"
confidence = (1 - prediction) if prediction < 0.5 else prediction

plt.subplot(1, num_demo, i + 1)
img_display = test_images[i]
img_display = img_display - img_display.min()
img_display = img_display / img_display.max()

plt.imshow(img_display)
plt.title(f'Истина: {true_label}\nПредск.: {predicted_class}\nУверен.: {confidence:.2%}')
plt.axis('off')

plt.tight_layout()
plt.show()

```



Создание полной модели:

```

inputs = keras.Input(shape=(IMG_HEIGHT, IMG_WIDTH, 3))
x = vgg_base(inputs, training=False) # VGG16 в режиме inference
x = layers.Flatten()(x)
x = layers.Dropout(0.5)(x)
x = layers.Dense(512, activation='relu')(x)
outputs = layers.Dense(1, activation='sigmoid')(x)

full_model = keras.Model(inputs, outputs)

full_model.compile(
    optimizer=keras.optimizers.RMSprop(learning_rate=1e-4),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Сохраняем полную модель
full_model.save('models/full_vgg_model.h5')

```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or  
`keras.saving.save_model(model)`. This file format is considered legacy. We  
recommend using instead the native Keras format, e.g.  
`model.save('my_model.keras')` or `keras.saving.save_model(model,  
'my_model.keras')`.
```

Вывод:

В ходе выполнения работы были последовательно исследованы три подхода к решению задач с разными датасетами. Первый этап продемонстрировал эффективность сверточных нейронных сетей для задач многоклассовой классификации. Обе модели достигли высокой точности 99% распознавания рукописных цифр, подтвердив преимущество сверточных операций перед полносвязными сетями в работе с пространственными данными. Второй этап выявил сложности обучения с нуля на ограниченных данных. Несмотря на применение аугментации и регуляризации, кастомные модели достигли точности 84% и 86%, что свидетельствует о необходимости более сложных архитектур и методов для решения нетривиальных задач бинарной классификации. Третий этап подтвердил эффективность переноса знаний при работе с малыми датасетами. Использование предобученной VGG16 в сочетании с простым классификатором позволило достичь точности 99%. Подход feature extraction значительно ускорил обучение и уменьшил вычислительные затраты.